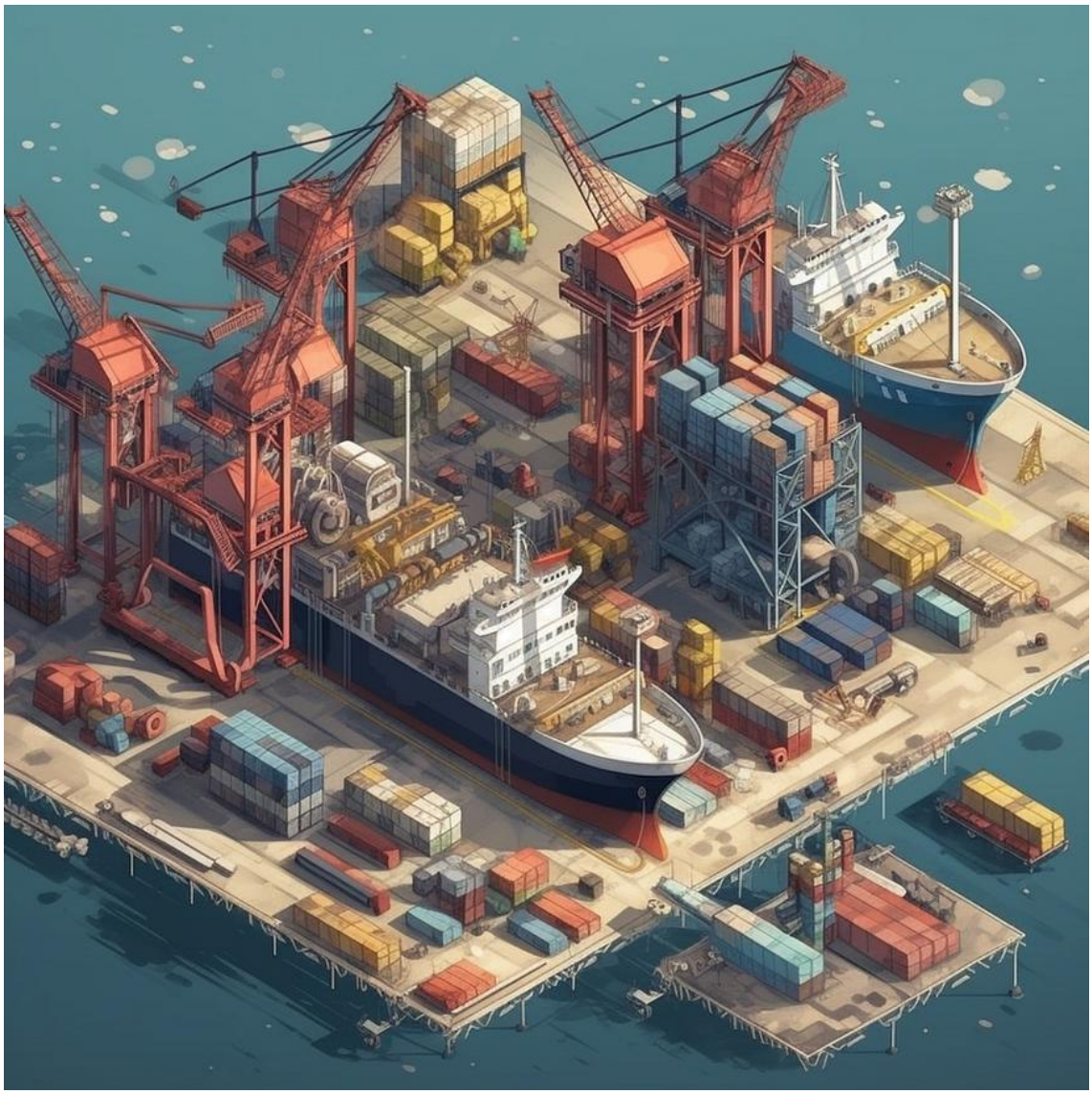
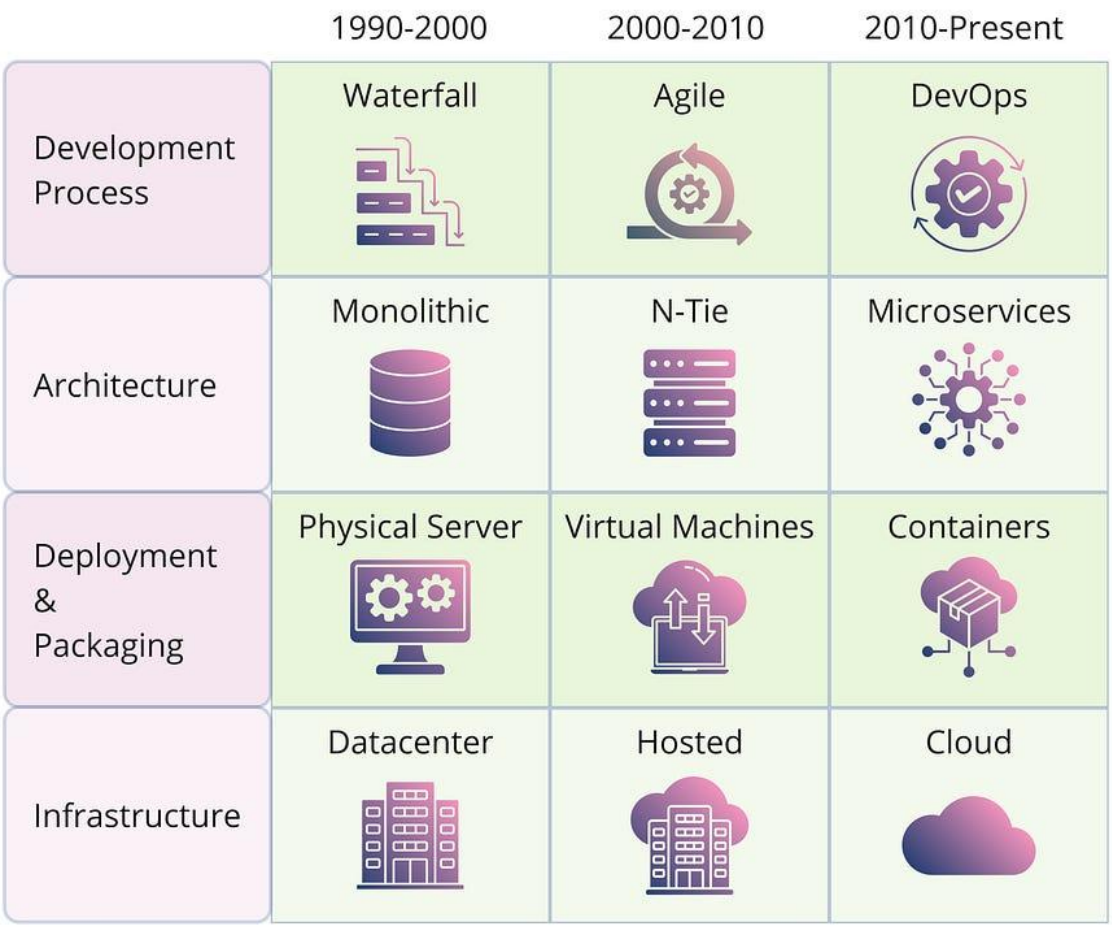
Contenerización: Docker y contenedores.



1. Desarrollo de software moderno y operaciones de TI
2. Introducción a los contenedores
3. Contenedores frente a máquinas virtuales
4. Docker
5. Otras tecnologías de contenedores
6. Caso de Podman

1. Desarrollo de software moderno y operaciones de TI

Las plataformas y los servicios en la nube constituyen la infraestructura esencial para DevOps y los microservicios, lo que permite el ecosistema moderno de desarrollo de software y operaciones de TI.



2. Introducción a los contenedores

La contenerización es un enfoque moderno para el desarrollo de software en el que una aplicación, junto con sus dependencias y configuración, se empaqueta en una sola unidad llamada imagen de contenedor. Este contenedor se puede probar e implementar fácilmente en cualquier sistema operativo (SO) anfitrión.

un paquete estandarizado y aislado que se utiliza para ejecutar aplicaciones y que incluye una aplicación y todas sus dependencias.

Una analogía sencilla = contenedores de envío

Piensa en los contenedores de software como si fueran contenedores de transporte marítimo. Así como los contenedores de transporte marítimo pueden llevar diferentes tipos de mercancías por barco, tren o camión, los contenedores de software pueden albergar distintos códigos y dependencias. Esto facilita a los desarrolladores y profesionales de TI la implementación de aplicaciones en diversos entornos con cambios mínimos.

Los contenedores también ayudan a mantener las aplicaciones aisladas entre sí en un sistema operativo compartido. Se ejecutan en un host de contenedores, que opera sobre el sistema operativo (Linux o Windows). Por ello, los contenedores son mucho más ligeros que las máquinas virtuales (VM).

Efímero, idempotente e inmutable

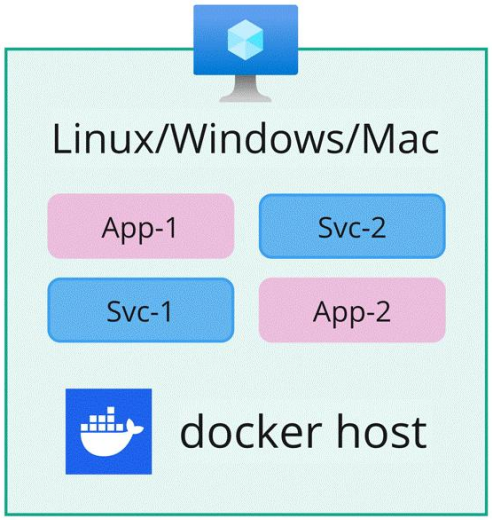
Los contenedores proporcionan a las aplicaciones y servicios que alojan las propiedades de efímero, idempotencia e inmutabilidad. En un entorno local de desarrollo o prueba, estas propiedades pueden parecer la peor opción; sin embargo, a gran escala, ofrecen escalabilidad y agilidad.

Efímero **=** *un proceso de corta duración*.

Idempotente **=** *produce el mismo resultado independientemente de cuántas veces se realice*.

Inmutable **=** *que no cambia con el tiempo o que no puede cambiarse*.

Cada contenedor puede ejecutar una aplicación web completa o un servicio. Por ejemplo, en una configuración de Docker, el host de Docker es el host del contenedor, y aplicaciones como App1, App2, Svc1 y Svc2 son los servicios en contenedores.



1. Efímeros: Los contenedores están diseñados para tener una vida útil corta y ser fácilmente reemplazables. Esto permite una rápida escalabilidad, actualizaciones ágiles y una utilización eficiente de los recursos, ya que los contenedores se pueden activar y desactivar según sea necesario sin un compromiso a largo plazo.
2. Idempotente: Las operaciones en contenedores se pueden repetir varias veces sin que cambie el resultado. Esto garantiza la consistencia y la fiabilidad, especialmente en sistemas distribuidos donde las operaciones podrían reintentarse debido a problemas de red.
3. Inmutable: Una vez creado, un contenedor no se modifica. Cualquier actualización o cambio implica el despliegue de un nuevo contenedor. Esto genera entornos más predecibles y estables, ya que el estado del contenedor permanece constante a lo largo de su ciclo de vida.

Ventajas de la contenerización

1. Abstracción: Los contenedores estandarizan las aplicaciones, haciéndolas independientes de los detalles del sistema anfitrión.
2. Escalabilidad: Amplíe fácilmente sus aplicaciones ejecutando múltiples contenedores.
3. Gestión de dependencias: Agrupe las aplicaciones con todas sus dependencias, simplificando la gestión de versiones.
4. Tolerancia a fallos: Aísla los fallos en contenedores individuales, mejorando así la resiliencia.
5. Seguridad: Aísle las aplicaciones para evitar la propagación de código malicioso.
6. Eficiencia de recursos: Comparta capas comunes para ahorrar espacio en disco.
7. Consistencia: Utilice archivos Docker para implementaciones predecibles y repetibles.

Casos de uso de la contenerización

1. Despliegue de microservicios: Entornos aislados para cada servicio, lo que permite una escalabilidad y gestión independientes.
2. Optimización del pipeline de CI/CD: Entornos consistentes en las etapas de desarrollo, pruebas y producción.

3. Modernización de aplicaciones: Facilita la implementación y la gestión de aplicaciones heredadas en entornos nativos de la nube.
4. Desarrollo y pruebas: Entornos reproducibles que replican las configuraciones de producción.
5. Edge Computing: Ejecutar las aplicaciones más cerca de la fuente de datos para reducir la latencia.
6. Procesamiento por lotes y de datos: Ejecución eficiente de tareas de datos a gran escala con escalabilidad horizontal.
7. Soluciones PaaS: Desarrollo y gestión de aplicaciones simplificados sin complejidades de infraestructura.

Resumen

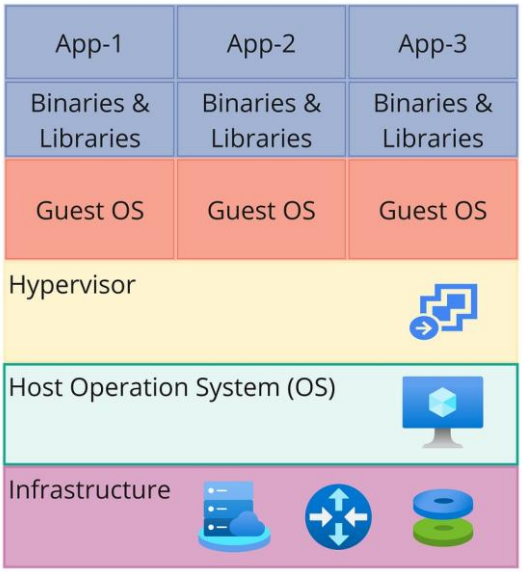
Un contenedor es una unidad de software estándar que agrupa el código y todas sus dependencias, lo que garantiza que la aplicación se ejecute de forma rápida y fiable en diferentes entornos informáticos. Una imagen de contenedor Docker es un paquete ejecutable, ligero e independiente que incluye todo lo necesario para ejecutar una aplicación: código, entorno de ejecución, herramientas del sistema, bibliotecas y configuración.

Los contenedores proporcionan aislamiento, portabilidad, agilidad, escalabilidad y control a lo largo de todo el ciclo de vida de la aplicación. El beneficio más significativo es el aislamiento entre los entornos de desarrollo (Dev) y de operaciones/producción (Ops/Prod).

3. Contenedores frente a máquinas virtuales

Máquinas virtuales (VM) = nivel de hardware

Las máquinas virtuales (VM) son como crear varios ordenadores a partir de una sola máquina física. Un software especial llamado hipervisor permite que varias VM se ejecuten en un único servidor. Cada VM tiene su propio sistema operativo, aplicaciones y todos los archivos necesarios, lo que puede ocupar mucho espacio (decenas de gigabytes). Además, las VM pueden tardar en arrancar.

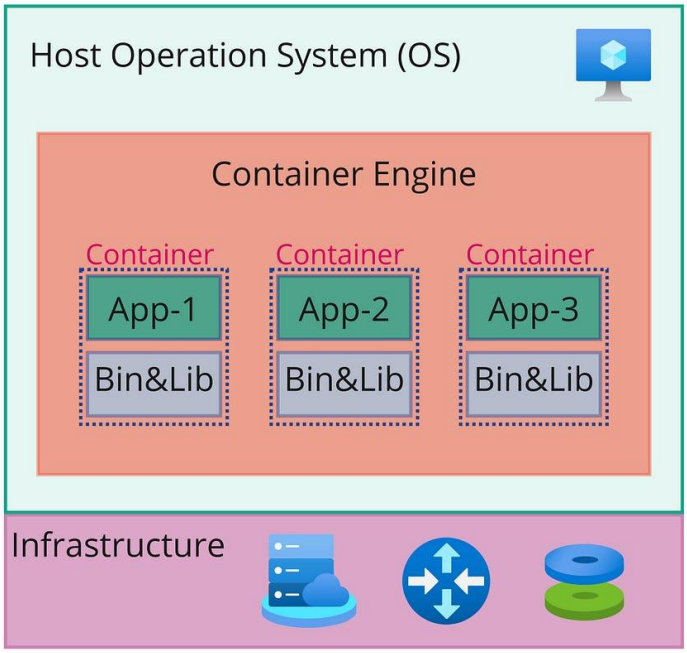


Arquitectura de máquinas virtuales

1. Capa de hardware: Esta es la base, que incluye la CPU, el almacenamiento y las interfaces de red.
2. Sistema operativo anfitrión e hipervisor: Por encima del hardware se encuentra el sistema operativo anfitrión y un hipervisor, que gestiona las máquinas virtuales.
3. Máquinas virtuales (VM): Cada máquina virtual incluye su propio sistema operativo, aplicaciones y archivos necesarios, y funciona como si fuera una máquina física independiente.
4. Aplicaciones y datos: Dentro de cada máquina virtual, encontrará las aplicaciones y los datos, aislados de otras máquinas virtuales en el mismo host.

Contenedores

Los contenedores son una forma de agrupar una aplicación y sus dependencias. Se ejecutan en la misma máquina y comparten el núcleo del sistema operativo, pero cada contenedor opera en su propio espacio aislado. Los contenedores son mucho más pequeños que las máquinas virtuales (generalmente solo unas decenas de megabytes), pueden ejecutar más aplicaciones y requieren menos máquinas virtuales y sistemas operativos (capas de abstracción).



Arquitectura de contenedores = Nivel del sistema operativo

1. Capa de hardware: Esta es la base, que incluye la CPU, el almacenamiento y las interfaces de red.
2. Sistema operativo anfitrión y núcleo: El sistema operativo anfitrión y su núcleo se encuentran por encima del hardware y gestionan las interacciones entre el software y el hardware.
3. Motor de contenedores: Esta capa, específica de la tecnología de contenedores, se ejecuta en el sistema operativo anfitrión.
4. Contenedores: En la parte superior se encuentran los contenedores, cada uno de los cuales contiene los binarios, las bibliotecas y las aplicaciones necesarias, todos ejecutándose en su propio espacio de usuario aislado.

4. Docker

Descripción general

Docker es una empresa famosa por crear una plataforma de contenedores (Docker) para desarrollar, distribuir y ejecutar aplicaciones utilizando tecnología de contenedores.

Evolución de la contenerización

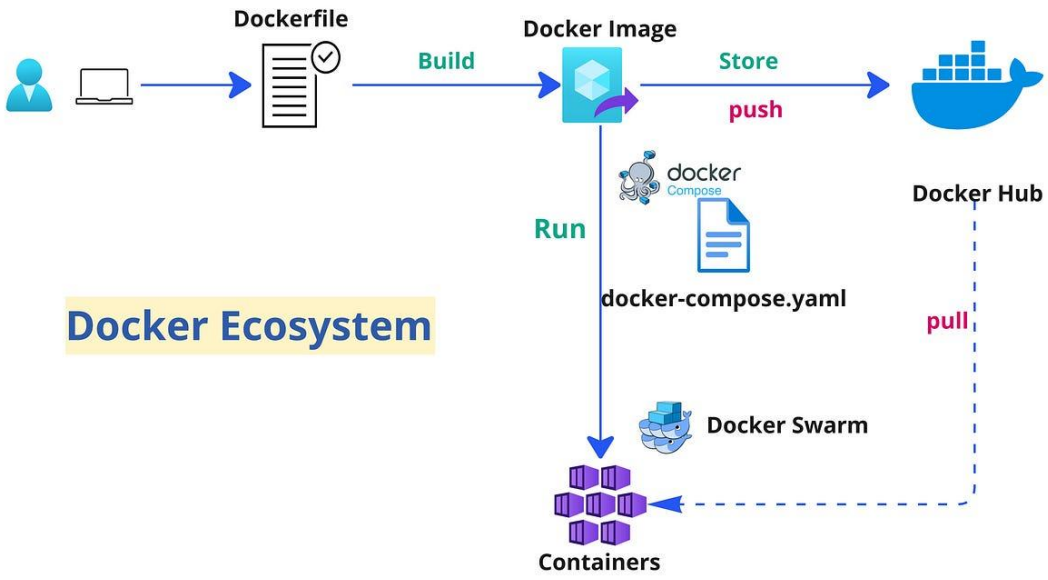
La contenerización comenzó con los cgroups de Linux, que gestionan y limitan el uso de recursos, y los contenedores de Linux (LXC), que proporcionan aislamiento para diferentes procesos. Los contenedores LXC incluyen los binarios y las bibliotecas necesarios para las aplicaciones, pero no empaquetan el núcleo del sistema operativo, lo que los hace ligeros y capaces de ejecutar muchas instancias en hardware limitado.

Contenedores Docker

Docker, basado en LXC, popularizó la gestión de contenedores y contribuyó a la creación de las especificaciones de la Open Container Initiative (OCI). Estos estándares garantizan la coherencia de las imágenes y los entornos de ejecución de los contenedores en diferentes entornos, lo que permite la compatibilidad multiplataforma, fundamental para los espacios de trabajo digitales modernos.

Al ejecutar una imagen de contenedor Docker en Docker Engine, esta se convierte en un contenedor. Los contenedores funcionan tanto en aplicaciones Linux como Windows y siempre se ejecutan de la misma manera, independientemente de la infraestructura subyacente. Aislan el software de su entorno, lo que garantiza un rendimiento constante a pesar de las diferencias entre los entornos de desarrollo y de pruebas.

Terminología y ciclo de vida de Docker



Ecosistema Docker

Dockerfile: Un Dockerfile es un archivo de texto con instrucciones para crear una imagen Docker. Imagínelo como una receta: la primera línea especifica la imagen

base, y las siguientes incluyen los pasos para instalar programas, copiar archivos y configurar el entorno.

```
# sintaxis=docker/dockerfile:1

 FROM node:lts-alpine
WORKDIR /app
COPY . .
RUN yarn install --production
CMD [ "node" , "src/index.js" ]
EXPOSE 3000
```

Compilación: La compilación es el proceso de crear una imagen de contenedor a partir del Dockerfile y los archivos adicionales que se encuentran en el directorio de compilación.

Imagen de contenedor: Una imagen de contenedor es un paquete que incluye todo lo necesario para crear un contenedor. Contiene todas las dependencias (como los frameworks) y la configuración para su despliegue y ejecución. Normalmente, una imagen se construye a partir de varias imágenes base apiladas para formar el sistema de archivos del contenedor. Una vez creada, la imagen no se puede modificar.

Contenedor: Un contenedor es una instancia de una imagen Docker que representa la ejecución de una única aplicación, proceso o servicio. Incluye el contenido de una imagen Docker, un entorno de ejecución y un conjunto estándar de instrucciones.

Volúmenes (almacenamiento): Los volúmenes proporcionan un sistema de archivos de escritura para los contenedores. Dado que las imágenes son de solo lectura (sin estado), pero la mayoría de los programas necesitan escribir datos, los volúmenes añaden una capa de escritura sobre la imagen del contenedor. De esta forma, los programas pueden escribir en el sistema de archivos como de costumbre. Los volúmenes se almacenan en el sistema anfitrión y Docker los gestiona.

Redes — Los tipos de red más comunes en Docker son:

- Red bridge: Cuando creas un contenedor sin especificar un controlador de red, este utiliza la red bridge predeterminada. Esto significa que el contenedor formará parte de una red interna privada en tu host Docker.
- Red host: En este modo, los contenedores comparten la pila de red del host. No obtienen sus propias direcciones IP, sino que utilizan la dirección

IP del host, lo que elimina el aislamiento de red entre el host Docker y los contenedores.

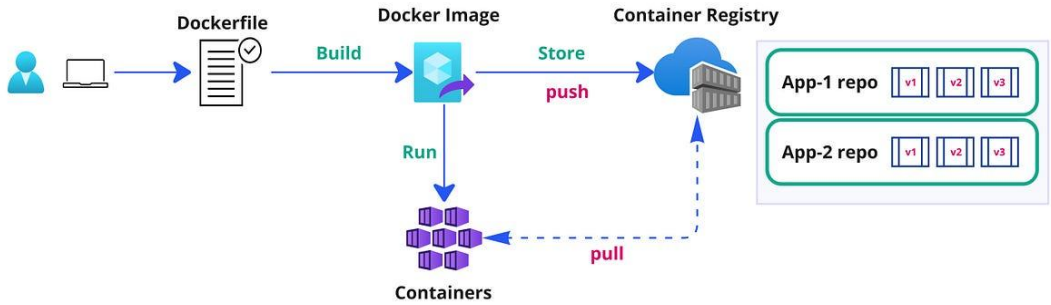
- Red overlay: Este tipo de red permite que los contenedores que se ejecutan en diferentes hosts de Docker se comuniquen entre sí. Se usa comúnmente en Docker Swarm para conectar servicios en varios hosts.

Etiqueta: Una etiqueta es un rótulo que se puede aplicar a las imágenes para identificar diferentes versiones o variantes de la misma imagen, según el número de versión o el entorno de destino.

Compilación en varias etapas: Esta función, disponible desde Docker 17.05, ayuda a reducir el tamaño de las imágenes finales. Por ejemplo, puede usar una imagen base grande con el SDK para compilar y publicar, y luego una imagen base más pequeña solo para ejecución para alojar la aplicación.

Registro de contenedores: Un registro es un servicio que proporciona acceso a repositorios. El registro predeterminado para la mayoría de las imágenes públicas es Docker Hub, propiedad de Docker. Los registros suelen contener repositorios de varios equipos. Las empresas a menudo tienen registros privados para almacenar y administrar sus propias imágenes. Algunos registros comunes son Docker Hub , Amazon ECR y Azure Container Registry .

Repositorio (repo): Un repositorio es una colección de imágenes Docker relacionadas, etiquetadas con etiquetas para indicar las versiones de las imágenes. Algunos repositorios contienen varias variantes de una imagen específica, como una con SDK (más pesada) y otra solo con entornos de ejecución (más ligera). Estas variantes se pueden marcar con etiquetas. Un mismo repositorio también puede contener variantes de plataforma, como una imagen para Linux y otra para Windows.



Docker Hub: Docker Hub es un registro público donde puedes subir y trabajar con imágenes de Docker.

Azure Container Registry: Este es un recurso público para trabajar con imágenes Docker en Azure. Proporciona un registro cercano a sus implementaciones de Azure y le permite controlar el acceso mediante grupos y permisos de Azure Active Directory.

Registro de confianza de Docker (DTR): DTR es un servicio de registro de Docker que se puede instalar localmente en el centro de datos y la red de una organización. Es útil para administrar imágenes privadas dentro de una empresa y está incluido en el producto Docker Datacenter.

Imagen multiarquitectura: Las imágenes multiarquitectura (o multiplataforma) son una característica de Docker que simplifica la selección de la imagen correcta para la plataforma en la que se ejecuta Docker. Por ejemplo, cuando un Dockerfile solicita una imagen base como mcr.microsoft.com/dotnet/sdk:8.0, selecciona automáticamente la versión apropiada para el sistema operativo y la versión.

Docker Desktop: Docker Desktop proporciona herramientas de desarrollo para Windows y macOS para crear, ejecutar y probar contenedores localmente. Es compatible con contenedores Linux y Windows.

Compose: Docker Compose es una herramienta de línea de comandos y un formato de archivo YAML para definir y ejecutar aplicaciones de múltiples contenedores. Se define una aplicación con múltiples imágenes mediante .ymlarchivos y, a continuación, se despliega toda la aplicación con un único comando (docker-compose up).

Clúster: Un clúster es un conjunto de hosts Docker que se presentan como un único host virtual Docker. Esto permite que las aplicaciones escalen en múltiples hosts. Los clústeres Docker se pueden crear con herramientas como Kubernetes, Docker Swarm, etc.

Orquestador: Un orquestador simplifica la gestión de clústeres y hosts Docker. Permite administrar imágenes, contenedores y hosts mediante una interfaz de línea de comandos (CLI) o una interfaz gráfica de usuario (GUI). Los orquestadores gestionan la red de contenedores, las configuraciones, el balanceo de carga, el descubrimiento de servicios, la alta disponibilidad y mucho más. El mejor ejemplo es Kubernetes.

5. Otras tecnologías de contenedores

Docker, como plataforma completa de contenedores, no es la única opción disponible. Docker cuenta con muchas alternativas que ofrecen diversas características y capacidades para la contenerización.

Aquí hay una lista de tecnologías por categoría con otras alternativas a la plataforma Docker, incluyendo motores de tiempo de ejecución, plataformas de orquestación, compilaciones de contenedores, etc.:

Tecnología de contenedores

- **Kaniko:** https://github.com/GoogleContainerTools/kaniko, Crea imágenes de contenedores k8s-native
- **Buildah:** https://buildah.io/, Construye contenedores
- **Docker:** https://www.docker.com/, El estándar de facto :)
- **Podman:** https://podman.io/, Ejecuta y desarrolla pods
- **Shipwright:** https://shipwright.io/, Puede utilizar diferentes motores, por ejemplo Kaniko.
- **Nerdctl:** https://github.com/containerd/nerdctl, Para compilación local, funciona bien con Rancher.
- **Skopeo:** https://github.com/containers/skopeo, Almacena y comparte, inspecciona imágenes remotas, mueve imágenes entre entornos, ejecútalo sin necesidad de root.
- **Rancher Desktop**
- **Imágenes de Chainguard:** https://images.chainguard.dev/ Imágenes base seguras y de tamaño reducido. «Desarrolle, distribuya y ejecute software seguro con imágenes de contenedores reforzadas».

Motores de tiempo de ejecución (cosas de bajo nivel)

- **CRI-O:** https://cri-o.io/
- **Containerd:** https://containerd.io/
- **Podman, Docker, Buildah**

6. Caso de Podman

Podman (Pod Manager) es una herramienta de código abierto desarrollada por Red Hat para crear, gestionar y ejecutar contenedores e imágenes compatibles con las especificaciones OCI (*Open Container Initiative*). Su principal diferencia con Docker es que **funciona sin un demonio central (*daemonless*)** y permite la ejecución en modo *rootless* (sin privilegios de administrador), lo que mejora la seguridad del sistema. Además, su interfaz de línea de comandos es casi idéntica a la de Docker (alias docker=podman).

Ejemplos con Contenedores en Podman

Gestionar un contenedor individual en Podman es muy similar a hacerlo en Docker.

Descargar y ejecutar un contenedor básico (ej. Nginx)

Ejecutar un servidor web Nginx en segundo plano

podman run -d --name mi-web -p 8080:80 nginx:latest

Ver los contenedores en ejecución

podman ps

Detener y eliminar el contenedor

podman stop mi-web

podman rm mi-web

Ejemplos con Pods en Podman

Una de las características exclusivas de Podman (heredada de Kubernetes) es el concepto de **pod**. Un pod es un grupo de uno o más contenedores que **comparten el mismo espacio de nombres de red, dirección IP y volúmenes**. Dentro de un pod, los contenedores se comunican entre sí directamente a través de localhost.

Crear un Pod vacío

podman pod create --name mi-pod -p 8080:80

Añadir contenedores dentro del Pod

Imagina que quieres un contenedor principal con una aplicación web y un contenedor "sidecar" (por ejemplo, un visor de logs o una base de datos ligera) que trabajen juntos:

1. Crear el pod exponiendo el puerto 8080 al exterior

podman pod create --name web-pod -p 8080:80

2. Agregar el contenedor de Nginx al pod

podman run -d --pod web-pod --name nginx-cont nginx:latest

3. Agregar un segundo contenedor (ej. BusyBox haciendo ping o un servicio auxiliar) al mismo pod

podman run -d --pod web-pod --name helper-cont busybox sleep 3600

Ver los pods y sus contenedores

podman pod ps

podman pod inspect web-pod

Detener y eliminar el Pod completo

Al eliminar el pod, se detienen y eliminan todos los contenedores que contiene:

podman pod rm -f web-pod